

Experiment 6 : Solve a constraint satisfaction problem such as map coloring, Sudoku or N-Queens using back tracking and consistency checking.

Objectives

To understand the concept of Constraint Satisfaction Problems (CSPs).

To implement a CSP solution using the backtracking algorithm.

To apply consistency checking for validating constraints during the search process.

To solve problems such as Map Coloring, Sudoku, or N-Queens efficiently.

To analyze the correctness and efficiency of backtracking with consistency checking in CSPs.

Procedure

- 1.Start the program.
- 2.Define the map using a dictionary where each region (A, B, C, D) stores its neighboring regions.
- 3.Define the available colors as Red, Green, and Blue.
- 4.Create the `is_consistent()` function to check whether the selected color for a region is different from the colors of all its neighboring regions.
- 5.Create the `solve_map()` function to solve the map coloring problem using backtracking.
- 6.Check whether all regions have been assigned colors. If yes, return the completed assignment as the solution.
- 7.Select the first unassigned region from the map.
- 8.Try each available color for the selected region.
- 9.Perform consistency checking using `is_consistent()`.

- 10.If the color satisfies all constraints, assign it to the region and recursively solve the remaining regions.
- 11.If no valid color is found for a region, remove the previous assignment (backtracking) and try another color.
- 12.Continue this process until all regions are assigned valid colors or no solution exists.
- 13.Display the final color assigned to each region.
- 14.Stop the program.

```
map_data={
    'A' :['B','C'],
    'B' :['A','C','D'],
    'C' :['A','B','D'],
    'D' :['B','C'],
}
colors = ['Red' , 'Green', 'Blue']
```

```
def is_consistent(region,color,assignment) :
    for neighbour in map_data[region] :
        if neighbour in assignment and assignment[neighbour] == color :
            return False
    return True
```

```
def solve_map(assignment) :
    if len(assignment) == len(map_data) :
        return assignment

    unassigned_region = None
    for region in map_data :
        if region not in assignment :
            unassigned_region = region
            break

    if unassigned_region is None:
        return assignment # All regions assigned
```

```

for color in colors :
    if is_consistent(unassigned_region,color,assignment) :
        assignment[unassigned_region] = color
        result = solve_map(assignment)
        if result is not None :
            return result
        del assignment[unassigned_region] # Backtrack
return None

```

```

solution = solve_map({})
print("Solution :")
if solution:
    for region,color in solution.items() :
        print(region,"=",color)
else:
    print("No solution found.")

```

```

Solution :
A = Red
B = Green
C = Blue
D = Red

```

Conclusion : The Map Coloring Constraint Satisfaction Problem was successfully solved using the backtracking algorithm with consistency checking. The program assigned valid colors to all regions while ensuring that no two adjacent regions had the same color, demonstrating how backtracking efficiently explores possible solutions and consistency checking prevents invalid assignments.

